Pointer-First Task Constitution and Governance Upgrade for Stela

1. Executive Summary

The **Stela** platform represents a paradigm shift in repository-resident artificial intelligence, moving from stochastic, prompt-based interactions to a deterministic, "Constitution-as-Code" governance model. This report provides an exhaustive architectural directive for elevating the **Task** (B-TASK) subsystem to parity with the already hardened **Agent** (R-AGENT) and **Skill** (S-LEARN) subsystems. The central thesis of this upgrade is the enforcement of a **"Pointer-First"** design philosophy: a Task must not contain ephemeral logic; it must serve as an immutable pointer to authorized execution paths, grounded in the Single Source of Truth (SSOT).

Current audits of the repository state reveal a critical governance gap. While Agents and Skills are subject to rigorous provenance tracking and "Anti-Drift" policing, the docs/library/tasks directory operates as a collection of loose narrative instructions. This violation of the "Filing Doctrine" introduces "Context Hazards"—ambiguities that allow AI Contractors to deviate from the Policy of Truth (PoT.md).

This document outlines the complete refactoring of the Task domain. It establishes the **Task Promotion Ledger** (docs/library/TASKS.md) as the central governance surface, defines the **Pointer-First Task Schema**, and specifies the automation logic for **Harvesting** and **Promotion** (ops/lib/scripts/task.sh). By enforcing "Linguistic Precision" and "Absolute Literalism," this upgrade ensures that Stela functions not merely as a coding assistant, but as a verifiable, self-correcting autonomous development platform.[1]

---

2. Constitutional Foundations and The Policy of Truth

To re-engineer the Task system, one must first deconstruct the "Physical Laws" that govern the Stela repository. These laws, codified in PoT.md, are not guidelines; they are constraints that define the system's reality.

2.1 The Doctrine of Non-Duplication

The Policy of Truth establishes the **SSOT Axiom**: "one canonical file per domain; other mentions are pointers".[1] This axiom is the primary defense against entropy. In a traditional repository, instructions on "how to test" might be duplicated in a README.md, a CONTRIBUTING.md, and a Jira ticket. In Stela, this duplication is classified as a "Drift" failure state.

The current state of docs/library/tasks represents a vulnerability in this doctrine. Legacy tasks often contain procedural steps (e.g., "Run npm test and check for errors") that duplicate logic found in ops/bin scripts or docs/MANUAL.md. If the testing script changes but the Task file does not, the Agent executes obsolete logic. This divergence is the root cause of "hallucination" in agentic workflows.[3]

2.2 The Pointer-First Paradigm

The "Pointer-First" philosophy draws upon computer science fundamentals where efficiency and accuracy are achieved by referencing memory addresses rather than copying large data structures.[4] Applied to AI governance, a "Pointer-First" Task does not describe *how* to do work; it describes *where* the authorized tools and knowledge reside.

- **Legacy Task (Narrative):** "Review the code for security vulnerabilities. Look for hardcoded passwords and ensure input validation is present." (Subjective, relies on AI training data, high risk of drift).
- **Pointer-First Task (Reference):** "Execute ops/bin/security-scan. Apply remediation patterns defined in docs/library/skills/S-LEARN-05. Enforce PoT.md Section 5.4." (Deterministic, anchored to SSOT, low risk of drift).

This shift transforms the Task from a "recipe" (which can be misread) into a "manifest" (which is verified). It enforces the **Behavioral Logic Standard** of "Absolute Literalism" by removing the ambiguity that necessitates interpretation.[1]

2.3 Staffing Protocols and Task Ownership

The staffing protocol defined in PoT.md assigns specific roles that interact with Tasks. The upgrade must enforce these roles through the Task lifecycle [1]:

Role	Responsibility	Interaction with Tasks
Operator (Human)	Final Authority	**Approves** Task promotions. Ensures that a Task added to the library represents a recurring, high-value workflow. Owns the TASKS.md ledger.
Integrator (Lead AI)	Governance & Audit	**Selects** and **Assigns** Tasks via Dispatch Packets (DPs). Relies on the Registry to identify available capabilities. Detects drift in

		Task execution.
Contractor (Guest AI)	Execution	**Executes** Tasks. Reads the B-TASK file to understand scope and required tools. **Harvests** new workflow patterns into draft Tasks.

---

3. Forensic Analysis of the Current State

A deep forensic analysis of the repository dump reveals a maturity gap between the Task subsystem and the rest of the platform.

3.1 The B-TASK Legacy Gap

The file list in dump-full-main-eeafcc36c.txt confirms the existence of docs/library/tasks/B-TASK-01.md through B-TASK-10.md.[1] However, contrast this with docs/library/agents/ and docs/library/skills/:

- **Agents (R-AGENT-XX):** These files have been refactored (DP-OPS-0031) to remove legacy metadata and use a "pointer-first" constitution. They contain explicit ## Provenance and ## Pointers sections.[1]
- **Skills (S-LEARN-XX):** These files are harvested via ops/lib/scripts/skill.sh, ensuring they contain strict "Drift Preventers" and "Friction Context".[1]
- **Tasks (B-TASK-XX):** There is no evidence of a corresponding task.sh script in ops/lib/scripts/. This implies that Tasks are currently created manually, likely containing narrative prose. This violates the "Automated Governance" directive.

Identified Weaknesses in Current Tasks:

1. **Lack of Provenance:** Legacy tasks lack a record of *when* they were created, *which* DP validated them, and *what* friction they address. Without provenance, an Agent cannot judge the "freshness" of the instruction.
2. **Narrative Overload:** Narrative instructions encourage "vibes-based" execution. "Check for style" is a vibe. "Run tools/lint/style.sh" is a law.
3. **Implicit Scope:** Without the explicit ## Scope Boundary section found in Agents, Tasks encourage Contractors to wander outside the bounds of the Dispatch Packet, violating the "Anti-Drift" directive.

3.2 The Registry Disconnect

The index lists docs/ops/registry/TASKS.md.[1] However, repo_gates.yml runs

tools/lint/library.sh. If the Task files do not adhere to a strict schema, the registry is merely a list of filenames, not a semantic verification tool. A robust system requires that the Registry acts as a "Gatekeeper"—if a file is not in the Registry, it does not exist to the System.[6]

3.3 The Missing "Harvest" Logic

The "Anti-Drift" governance requires that the system's capabilities evolve naturally from its operations ("Equilibrium Maintenance"). Currently, ops/lib/scripts/agent.sh and skill.sh automate this for agents and skills. The absence of task.sh breaks the feedback loop for Tasks. Valuable workflows discovered during a session are lost or must be manually transcribed, introducing human error.[7]

---

4. The New Task Constitution: docs/library/TASKS.md

To align Tasks with the governance standards of Stela, we must establish a central ledger similar to AGENTS.md and SKILLS.md. This file, docs/library/TASKS.md, will serve as the **Promotion Ledger** for tasks.

4.1 Purpose and Doctrine

The TASKS.md file is not a table of contents; it is a transactional log of governance events. It records the transition of a "workflow" (an ephemeral sequence of actions in a DP) into a "Task" (a codified, reusable canon artifact).

Doctrine of Task Promotion:

- **Provenance Required:** A Task is only promoted if it has been successfully executed in a prior Dispatch Packet (DP). Theoretical tasks are forbidden.
- **Reuse-First:** A Task must address a recurring need. If a template exists in ops/, it must be used.
- **Orchestration, Not Logic:** A Task must not duplicate the logic of a Skill or Agent; it must *orchestrate* them.

4.2 The Promotion Packet Schema

The core of the new constitution is the **Promotion Packet**. This template defines the metadata required to promote a task. This structure mirrors the rigorous validation used for Agents and Skills.[1]

Proposed Structure for docs/library/TASKS.md

Task Promotion Ledger

> **CONTEXT HAZARD:** Tasks are JIT-only. Do not add task libraries to

ops/lib/manifests/CONTEXT.md.

Logging to docs/library/TASKS.md is operator-mediated.

This file is the operator-facing promotion workflow for canon tasks. Use ops/lib/scripts/task.sh to append candidates and promotion entries.

Harvest Engine Workflow

- ops/lib/scripts/task.sh harvest creates a draft in storage/handoff/ with provenance.
- Review and refine the draft. Do not edit the Provenance block.
- ops/lib/scripts/task.sh promote moves the draft to docs/library/tasks/ and updates the registry.
- ops/lib/scripts/task.sh check audits scope boundaries and toolchain validity.

Promotion Packet Template

Promotion Packet: Task Promotion

- **Required fields:**
  - **Task Name:** Canonical name used in registry (e.g., audit-security).
  - **Task ID:** B-TASK-XX (Sequential).
  - **Provenance:** Source DP ID, Branch, HEAD.
  - **Objective:** One sentence goal (must match TASK.md goal format).
- **Pointer Set:**
  - **Authorized Toolchain:** List of ops/bin/ executables.
  - **Required Agents:** List of R-AGENT-XX IDs.
  - **JIT Skills:** List of S-LEARN-XX IDs.
  - **Reference Docs:** Pointers to docs/MANUAL.md or docs/specs/.
- **Drift Preventers:**
  - **Scope Boundary:** Explicit "Do Not Touch" zones.
  - **Stop Conditions:** Logical gates that force an abort (e.g., "Stop if linter fails").
- **Definition of Done:**
  - SoP.md updated.
  - docs/ops/registry/TASKS.md updated.
  - Proof bundle (RESULTS.md) verified.

Candidate Log (append-only)

(Automated entries from harvest script)

Promotion Log (append-only)

(Automated entries from promote script)

4.3 Structural Analysis of the Schema

- **Context Hazard Warning:** This is critical. Tasks, like Agents, are "heavy" context. They should not be loaded into the global context window (CONTEXT.md). They are loaded **Just-In-Time (JIT)** when referenced by a DP. This prevents context window saturation and reduces costs.[1]
- **Provenance:** By tying every task to a DP ID, we create an audit trail. If B-TASK-05 fails in the future, we can trace it back to DP-OPS-00XX to see the original context in which it succeeded. This enables "forensic debuggability".[9]
- **Orchestration Pointers:** Unlike an Agent (which *is* a role) or a Skill (which *is* knowledge), a Task is an *action plan*. It effectively says: "Use Agent X, equipped with Skill Y, running Binary Z."

---

5. Reformatting B-TASK-XX: The Pointer-First Standard

The existing B-TASK files must be refactored to strip out narrative prose and replace it with hard pointers. This section defines the "Pointer-First Task" format and applies it to the existing task list.

5.1 The Logic of "Zero-Hallucination"

A narrative instruction like "Review the code for style" allows the AI to hallucinate what "style" means. A pointer instruction like "Run tools/lint/style.sh" leaves no room for interpretation. If the script passes, the style is correct. If it fails, it is wrong. This binary state is essential for "Anti-Drift Governance."

5.2 The New B-TASK Template

Every B-TASK file must strictly adhere to this structure:

Task: [Name]

Provenance

- **Captured:**
- **DP-ID:**
- **Branch:**
- **HEAD:** [Commit Hash]

- **Objective:** [Original Objective]

Orchestration

- **Primary Agent:** (The ideal persona for this task)
- **Supporting Agents:** [List or None]

Pointers (SSOT)

- **Constitution:** PoT.md
- **Governance:** docs/GOVERNANCE.md
- **Contract:** TASK.md
- **Tools:**
  - [path/to/binary]
  - [path/to/script]
- **Skills (JIT):**
  - ``

Execution Logic

1. **Pre-Flight:** Verify `` is executable.
2. **Execution:** Run [Command].
3. **Verification:** Check [Output] against ``.
4. **Correction:** If fail, apply ``.

Scope Boundary

- **Allowed:**
- **Forbidden:**
- **Stop Condition:**

5.3 Reconstruction of Existing Tasks (B-TASK-01 to B-TASK-10)

Based on the file list in dump-full-main-eeafcc36c.txt [1] and the analysis of standard repository operations, we reconstruct the tasks using the Pointer-First standard.

B-TASK-01: Code Review (Refactor)

- **Orchestration:** R-AGENT-02 (Code Reviewer).
- **Tools:** tools/lint/style.sh, tools/lint/truth.sh.
- **Skills:** S-LEARN-04 (Coding Standards), S-LEARN-05 (Security Protocols).
- **Logic:**
  1. Execute git diff --name-only.
  2. Execute tools/lint/style.sh. If fail, STOP.

3. Execute tools/lint/truth.sh. If fail, STOP.
4. Apply S-LEARN-04 patterns to diff. Report deviations quantitatively (e.g., "3 functions exceed 50 lines").
5. Apply S-LEARN-05 to check for secrets/injection.

B-TASK-02: Verification Command (Refactor)

- **Orchestration:** R-AGENT-02 (Code Reviewer) or R-AGENT-06 (Security).
- **Tools:** tools/verify.sh.
- **Skills:** S-LEARN-01 (Verification Loop).
- **Logic:**
  1. Confirm clean porcelain (git status).
  2. Execute tools/verify.sh.
  3. Capture output to RESULTS.
  4. Assert exit code is 0.

B-TASK-03: Integrate Command (Refactor)

- **Orchestration:** R-AGENT-04 (Integrator).
- **Pointers:** PoT.md (Section 5.2 Standard Workflow), .github/workflows/repo_gates.yml.
- **Logic:**
  1. Verify SoP.md updated in the PR.
  2. Verify repo-gates passed.
  3. Verify TASK.md updated if gate surfaces changed (task_policing.yml).
  4. Generate "Mandatory Closing Block" for PR description.

B-TASK-04: Plan Command (Refactor)

- **Orchestration:** R-AGENT-01 (Architect).
- **Tools:** ops/bin/map, ops/bin/llms --profile=architect.
- **Skills:** S-LEARN-02 (Project Guidelines).
- **Logic:**
  1. Read docs/MAP.md to understand terrain.
  2. Generate implementation plan referencing existing patterns.
  3. Output plan to storage/handoff/PLAN-.md.

B-TASK-05: Refactor Clean (Refactor)

- **Orchestration:** R-AGENT-05 (Refactor Cleaner).
- **Tools:** tools/verify.sh.
- **Skills:** S-LEARN-06 (Git Forensics).
- **Logic:**
  1. Identify dead code via S-LEARN-06.
  2. Delete code.
  3. Run tools/verify.sh. If fail, REVERT immediately.

B-TASK-06: Update Documentation (Refactor)

- **Orchestration:** R-AGENT-03 (Doc Updater).
- **Tools:** ops/bin/map (to refresh map), ops/bin/help.
- **Logic:**
  1. Identify changed surfaces via git diff --name-only.
  2. Cross-reference with docs/MAP.md.
  3. Update corresponding docs/ files to match new behavior.
  4. Run ops/bin/map to auto-generate index.

B-TASK-07: Refresh + Discuss (Analyst Stance)

- **Orchestration:** R-AGENT-01 (Architect).
- **Tools:** ops/bin/open, ops/bin/dump.
- **Logic:**
  1. Read OPEN artifact.
  2. Read dump artifact.
  3. Output analysis ONLY. No file edits permitted.

B-TASK-08: Refresh + Draft DP (Architect Stance)

- **Orchestration:** R-AGENT-04 (Integrator).
- **Pointers:** TASK.md (Template).
- **Logic:**
  1. Read OPEN and dump.
  2. Draft DP-OPS-XX using strict TASK.md headings.
  3. Output to storage/dp/intake/.

B-TASK-09: Refresh + Conform DP (Hygiene Stance)

- **Orchestration:** R-AGENT-04 (Integrator).
- **Tools:** tools/lint/dp.sh.
- **Logic:**
  1. Read legacy DP.
  2. Apply current TASK.md headers.
  3. Verify via tools/lint/dp.sh.

B-TASK-10: Refresh + Audit (Gatekeeper Stance)

- **Orchestration:** R-AGENT-04 (Integrator).
- **Tools:** tools/lint/context.sh, tools/lint/truth.sh.
- **Logic:**
  1. Read RESULTS.md.
  2. Verify RECEIPT contains OPEN, PORCELAIN, DUMP.
  3. Verify verification steps were RUN.

4. Output PASS/FAIL decision.

---

6. Building Operational Rigor: The Automation Layer

To elevate TASKS to the maturity level of AGENTS and SKILLS, manual creation must be deprecated. We must implement the **Immune System** for tasks through automation.

6.1 ops/lib/scripts/task.sh (The Harvest/Promote Engine)

This script will be the only authorized way to create or modify tasks, ensuring provenance is never forged.

Script Logic Specification:

Bash

```
#!/usr/bin/env bash
# ops/lib/scripts/task.sh (Pseudo-logic)

function harvest() {
  # Input: --name, --dp, --objective
  # Action: Scans TASK.md for the active "Execution Plan".
  # Capture: git metadata (Branch, Hash, Timestamp).
  # Output: storage/handoff/task-draft--[slug].md
  # Content: Injects the "Pointer-First" template with captured metadata.
}

function promote() {
  # Input: <draft_path>
  # Validation: Checks for placeholders (TODO, TBD) and validates pointers exist.
  # ID Assignment: Scans docs/library/tasks/ for highest B-TASK-XX, increments +1.
  # Move: Moves draft to docs/library/tasks/B-TASK-XX.md.
  # Registry: Appends row to docs/ops/registry/TASKS.md.
  # Ledger: Appends log entry to docs/library/TASKS.md.
}

function check() {
  # Action: Audits all B-TASK files.
  # Checks: Do pointers resolve? Is the Agent ID valid? Is it in the Registry?
```

```
}
```

6.2 tools/lint/task.sh (The Linter)

This linter will integrate into repo_gates.yml to prevent corrupt tasks from entering main.

Logic Gates:

1. **Filename Gate:** Must match B-TASK-[0-9]{2}.md.
2. **Structure Gate:** Must contain ## Provenance, ## Pointers, ## Scope Boundary.
3. **Pointer Gate:** All referenced files in ## Pointers must exist on disk.
4. **Registry Gate:** The file must be present in docs/ops/registry/TASKS.md.
5. **Linguistic Gate:** grep -E "don't|can't" (Fail if found).

6.3 Integration with repo_gates.yml

The existing repo_gates.yml runs tools/lint/library.sh. We must update library.sh to include a call to tools/lint/task.sh. This ensures that a corrupt or non-compliant Task file prevents a Pull Request from merging. This makes the governance rules **Physical Laws** rather than suggestions.[1]

---

7. Integration: The Agent-Task-Skill Triad

The true power of the Stela system emerges when Agents, Tasks, and Skills are orthogonal but interlocking entities. This combinatorial model allows for exponential capability with linear complexity.

7.1 The Combinatorial Model

Entity	Definition	Example
Agent (Who)	A specialized persona (R-AGENT) with a defined scope, tool access, and context constraints.	R-AGENT-06 (Security Reviewer)
Task (What)	A sequence of pointers defining a unit of work. It orchestrates Agents and Tools.	B-TASK-11 (Audit Dependencies)
Skill (How)	A reusable heuristic or	S-LEARN-05 (Security

	knowledge pattern for solving a specific friction point.	Protocols)

7.2 Dynamic Composition Examples

Scenario A: Routine Security Audit

- **Configuration:** R-AGENT-06 executes B-TASK-11.
- **Flow:** The Security Agent follows the Task pointers to run ops/bin/security-scan. It encounters an error. It loads S-LEARN-05 (JIT) to interpret the error. It fixes the issue.
- **Result:** High fidelity, low hallucination.

Scenario B: Architectural Security Review

- **Configuration:** R-AGENT-01 (Architect) executes B-TASK-11.
- **Flow:** The Architect runs the same Task (B-TASK-11) but interprets the results through a different lens (System Design vs. Code Implementation).
- **Result:** The same *Task* serves multiple *Agents* differently, creating leverage.

Scenario C: Skill Upgrade

- **Action:** We improve S-LEARN-05 with new findings on SQL Injection.
- **Impact:** Immediately, *every* Task (B-TASK-01, B-TASK-11) and *every* Agent (R-AGENT-02, R-AGENT-06) that points to that Skill becomes smarter. No refactoring of Tasks or Agents is required.

7.3 Hierarchical Patterns

This model supports hierarchical execution.[10]

- **Level 1:** R-AGENT-04 (Integrator) reads TASK.md (The Meta-Task).
- **Level 2:** Integrator assigns B-TASK-01 to R-AGENT-02.
- **Level 3:** R-AGENT-02 loads S-LEARN-04 to execute the task.

This hierarchy ensures that high-level strategy (Integrator) is decoupled from low-level execution (Contractor), maintaining "Equilibrium".[1]

---

8. Security & Context Hygiene

The "Pointer-First" upgrade directly addresses the "Context Hazard" vulnerabilities identified in the dump.

8.1 JIT Context Loading

PoT.md explicitly forbids loading docs/library/ into the global CONTEXT.md.[1] This is because libraries are vast. By using Pointers, we enable **Just-In-Time (JIT) Loading**.

- **Wrong:** Load all tasks at session start (Context saturation, high cost).
- **Right:** Load TASK.md. It points to B-TASK-01. Worker reads B-TASK-01. It points to S-LEARN-04. Worker reads S-LEARN-04.
- **Result:** The Context Window contains *only* what is necessary for the immediate nanosecond of execution.

8.2 Anti-Drift and Hallucination Control

Hallucinations occur when an AI guesses at a rule.

- **Narrative Task:** "Check for valid variable names." (AI guesses the convention).
- **Pointer Task:** "Run lint --check-names." (Binary outcome).
  By enforcing the use of ops/bin tools within Tasks, we force the AI to rely on deterministic code rather than probabilistic generation.

---

9. Implementation Roadmap

The execution of this upgrade should follow a standard Dispatch Packet structure, ensuring no disruption to ongoing operations.

9.1 Phase 1: Infrastructure (The Scaffold)

- **Objective:** Establish the governance container.
- **Steps:**
  1. Create docs/library/TASKS.md using the template defined in Section 4.2.
  2. Draft ops/lib/scripts/task.sh (harvest/promote logic).
  3. Draft tools/lint/task.sh (validation logic).
  4. **Verification:** Run task.sh check to ensure the empty state is valid.

9.2 Phase 2: Refactoring (The Migration)

- **Objective:** Convert legacy logic to pointers.
- **Steps:**
  1. Iterate through B-TASK-01 to B-TASK-10.
  2. Rewrite each using the "Pointer-First" template (Section 5.2).
  3. Ensure every "Execution Logic" step points to a file in ops/bin or tools/.
  4. **Registry:** Update docs/ops/registry/TASKS.md to match the new metadata.

9.3 Phase 3: Enforcement (The Gate)

- **Objective:** Lock the door behind us.
- **Steps:**
  1. Update tools/lint/library.sh to include task.sh checks.
  2. Update .github/workflows/repo_gates.yml if necessary.
  3. **Verification:** Attempt to commit a "bad" task (narrative only) and confirm repo-gates fails.

9.4 Phase 4: Operational Drill (The Test)

- **Objective:** Verify the harvesting loop.
- **Steps:**
  1. Execute a dummy DP that requires "Task Harvesting."
  2. Run ops/lib/scripts/task.sh harvest.
  3. Verify the draft appears in storage/handoff/.
  4. Run ops/lib/scripts/task.sh promote.
  5. Verify the new task appears in docs/library/tasks/ and the Registry.

---

10. Conclusion

The transition to a **Pointer-First Task Constitution** is a critical maturity milestone for the Stela platform. It represents the final step in closing the governance loop, bringing the **Task** entity up to the rigorous standards of **Agents** and **Skills**.

By stripping Tasks of narrative ambiguity and replacing them with executable pointers, we achieve three strategic goals:

1. **Determinism:** We replace "vibes" with binary execution.
2. **Auditability:** We replace "magic" with provenance-backed history.
3. **Efficiency:** We replace context saturation with JIT loading.

This architecture ensures that Stela remains a low-entropy, high-trust environment where the "Authority of the Human" and the "Logic of the AI" [1] are mediated by a rigid, verifiable constitution. The system does not just *do* work; it *understands* how the work is defined, recorded, and evolved.

End of Report.

Appendix: Task Registry Table Update (Hypothetical)

To satisfy the requirement of updating docs/ops/registry/TASKS.md, the new content should look like this:

ID	Name	Provenance (DP)	Roles
B-TASK-01	Code Review	DP-OPS-MIGRATION	R-AGENT-02
B-TASK-02	Verification Command	DP-OPS-MIGRATION	R-AGENT-02
B-TASK-03	Integrate Command	DP-OPS-MIGRATION	R-AGENT-04
B-TASK-04	Plan Command	DP-OPS-MIGRATION	R-AGENT-01
B-TASK-05	Refactor Clean	DP-OPS-MIGRATION	R-AGENT-05
B-TASK-06	Update Documentation	DP-OPS-MIGRATION	R-AGENT-03
B-TASK-07	Refresh + Discuss	DP-OPS-MIGRATION	R-AGENT-01
B-TASK-08	Refresh + Draft DP	DP-OPS-MIGRATION	R-AGENT-04
B-TASK-09	Refresh + Conform DP	DP-OPS-MIGRATION	R-AGENT-04
B-TASK-10	Refresh + Audit	DP-OPS-MIGRATION	R-AGENT-04

This registry acts as the single source of truth for the Integrator agent when selecting tools for a Dispatch Packet.

Works cited

1. dump-full-main-eeafcc36c.txt
2. Law-Following AI: Designing AI Agents to Obey Human Laws - The Fordham Law Archive of Scholarship and History, accessed February 7, 2026,

https://ir.lawnet.fordham.edu/cgi/viewcontent.cgi?article=6171&context=flr
3. AutoAgents: A Framework for Automatic Agent Generation - arXiv, accessed February 7, 2026, https://arxiv.org/html/2309.17288v3
4. Computer Science Fundamentals: Pointers | by Curt Corginia - Medium, accessed February 7, 2026, https://corgicorporation.medium.com/computer-science-fundamentals-pointers-bf9aa6f3f55c
5. Pointer (computer programming) - Wikipedia, accessed February 7, 2026, https://en.wikipedia.org/wiki/Pointer_(computer_programming)
6. Governance and security for AI agents across the organization - Cloud Adoption Framework | Microsoft Learn, accessed February 7, 2026, https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ai-agents/governance-security-across-organization
7. One year of agentic AI: Six lessons from the people doing the work - McKinsey, accessed February 7, 2026, https://www.mckinsey.com/capabilities/quantumblack/our-insights/one-year-of-agentic-ai-six-lessons-from-the-people-doing-the-work
8. Spring AI Agentic Patterns (Part 1): Agent Skills - Modular, Reusable Capabilities, accessed February 7, 2026, https://spring.io/blog/2026/01/13/spring-ai-generic-agent-skills/
9. LSST DM Metadata and Provenance - GitHub, accessed February 7, 2026, https://github.com/lsst-dm/provenance_proto/blob/master/Provenance.md
10. What are Hierarchical AI Agents? - IBM, accessed February 7, 2026, https://www.ibm.com/think/topics/hierarchical-ai-agents